

AUTOMATIZACIÓN Y DESPLIEGUE DE UNA APLICACIÓN WEB CON KUBERNETES, K3S Y ANSIBLE



Universidad
Francisco de
Vitoria

UFV Madrid

PABLO PÉREZ HERRERO

**2º CURSO ADMINISTRACIÓN DE SISTEMAS INFORMÁTICOS EN
RED**

INDICE

1.	ABSTRACT	4
2.	RESUMEN Y OBJETIVOS	4
3.	ANTECEDENTES	4
4.	ANÁLISIS Y ESPECIFICACIÓN DE REQUISITOS	5
5.	PROPUESTA DE SOLUCIÓN	5
6.	PLAN DE TRABAJO	5
7.	DESARROLLO DE LA SOLUCIÓN	6
8.	DESPLIEGUE E INSTALACIÓN.....	6
9.	EVOLUCIÓN Y TRABAJO FUTURO	7
10.	BIBLIOGRAFÍA.....	7

1. ABSTRACT

This Project studies the design and implementation of automated deployment platforms using modern DevOps methods and container-oriented technologies. The solution is based on a lightweight Kubernetes distribution (k3s) deployed over a multi-node cluster and fully provisioned through Ansible, which enables scalation on the infrastructure. The application is based on Python, with Flask and SQLite containerized with Docker and orchestrated through Kubernetes objects as Deployments, Services, Secrets, and Ingress. The objective of this project is to provide a reliable, automated and maintainable environment that reflect real-world practices used on IT operations.

2. RESUMEN Y OBJETIVOS

El objetivo del proyecto es presentar una solución a la necesidad de alta disponibilidad de la mayoría de aplicaciones Web. Lo conseguiremos mediante una distribución ligera de Kubernetes (k3s) y Ansible, aunque esta parte es simplemente a nivel local, como demostración. El proyecto constará de dos partes bien definidas:

- Despliegue LOCAL de una app de 3 clúster balanceados, desplegándolos y configurándolos de forma automatizada para su escalabilidad. Comprobaremos y demostraremos como, con menos clusters, la disponibilidad es mucho mejor.
- Mismo despliegue pero en AWS, con prácticas DevOps como CI/CD, sustituyendo k3s por la misma aplicación de Amazon (EKS), igual con la base de datos que en este caso será Amazon RDS en vez de SQLite. Todo esto se implementaría a través de un pipeline CI/CD, con GitHub Actions o GitLab, que automatice todo el proceso, desde que se sube el código hasta que se despliegue una nueva versión de la aplicación.

3. ANTECEDENTES

Esta solución, aplicada en prácticamente cualquier despliegue de aplicación hoy en día (por supuesto, en el mundo profesional, son despliegues mucho más grandes y sofisticados), surge de la necesidad de despliegues automáticos y alta disponibilidad. Desplegar una aplicación manualmente, una vez por cada nodo de balanceo, es algo terriblemente ineficiente y que puede dar lugar a múltiples errores de todo tipo; cada paso manual es una oportunidad para crear incoherencias e inconsistencias entre las distintas fases del despliegue.

Cada una de las aplicaciones que se usan en el despliegue cubren una necesidad:

- **Docker:** Garantiza que tu aplicación web se despliegue siempre de la misma manera, dentro de un container, y siguiendo exactamente el mismo proceso, con la ventaja de que es una mini-virtualización securizada y aislada. Hace que funcione igual en cualquier sistema.
- **Kubernetes:** Permite que cada pod acceda a otro sin conocer su IP real, balancea los pods y realmente siempre estás accediendo al mismo servicio aunque sean diferentes pods. Además, incluye configuración SQL de forma nativa, lo cual facilita muchísimo el despliegue a nivel global, con el extra de tener Traefik (Ingress), que se puede usar para el reverse proxy, permitiendo acceder a la app desde fuera del cluster de forma segura. En nuestro caso, en el despliegue local, usaremos k3s, que es una versión ligera de Kubernetes.
- **Ansible:** Evita que tengamos que realizar configuraciones manuales en los servidores, asegura que todos los pods estén idénticamente configurados en el clúster y puede volver a levantar el mismo servicio de forma rápida y eficaz. Instala todas las dependencias, configura SSH para que el pod master hable con los workers, configura IPs, firewalls...

4. ANÁLISIS Y ESPECIFICACIÓN DE REQUISITOS

5. PROPUESTA DE SOLUCIÓN

6. PLAN DE TRABAJO

El plan de trabajo será el siguiente:

1. Crear un repositorio GIT y su correspondiente GitHub, para llevar una cronología del proyecto y tener una copia online, desacoplada del local (por si acaso).
2. Instalación y configuración de Máquinas virtuales. Creamos 3 máquinas virtuales, un master y dos workers, y le damos IP en la misma red local. Al master le damos salida a internet. Los workers realmente no necesitarían para el funcionamiento del servicio, pero sí para instalar los paquetes. Configuramos también el SSH y la clave desde el master a los workers.
3. Creamos el Inventario Ansible, y aplicamos los roles a cada una de las máquinas.
4. Instalamos k3s de forma automatizada.
5. Desarrollamos la aplicación, el dockerfile y la imagen de la aplicación.

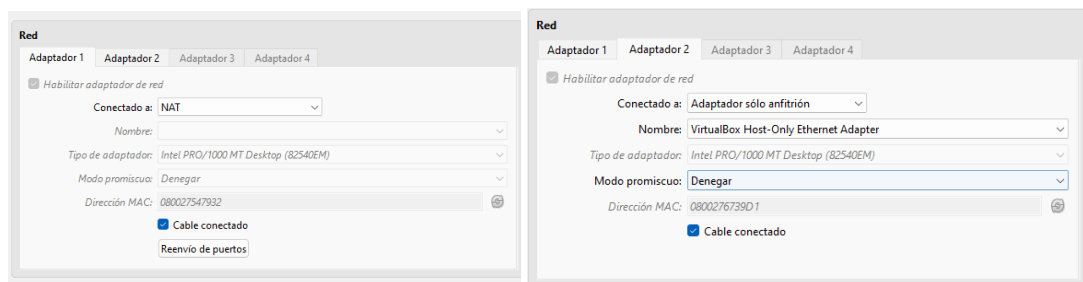
6. Desplegamos en Kubernetes (k3s).
7. Probamos su funcionamiento, su rendimiento y damos los ajustes finales.

7. DESARROLLO DE LA SOLUCIÓN

8. DESPLIEGUE E INSTALACIÓN

1. CREACIÓN DE MÁQUINA VIRTUAL MASTER EN VIRTUAL BOX Y WORKERS

Lo primero es crear la máquina virtual Master en VirtualBox, que será la referencia para los dos Workers. Elegiremos Ubuntu server como distro, ya que es ligera y fácil de instalar y usar. Es muy importante que la máquina tenga dos adaptadores de red, el primero “red NAT” para salida a internet y el segundo “Solo Anfitrión” con una red que hemos preconfigurado, siendo 192.168.56.0/24.



Procedemos con la instalación de Ubuntu Server, una vez hemos creado el usuario y lo hemos instalado, tenemos que cambiarle el hostname a k8s-master (a partir de ahora, cada vez que me refiera a k8s, es un slang para Kubernetes). Por último, modificamos el adaptador de red enp0s8, que es el de la red local, con el netplan, poniéndole su IP como 192.168.56.10.

Teniendo ya configurado el Master, lo más fácil es clonarlo para los dos workers, reiniciar sus MAC, cambiarles sus hostnames¹ a k8s-worker1 y k8s-worker2, cambiarles sus IPs a la 11 y la 12, y añadir las tres máquinas virtuales a los hostnames, para que podamos verlas sin tener que poner sus IPs; y ya tendríamos nuestro entorno virtual de trabajo creado, en menos de 15 minutos.

¹ Hago hincapié en los hostnames, porque son cruciales a la hora de desplegar Kubernetes y Ansible. Si no están bien puestos, no va a funcionar nada y los pods no se van a ver entre ellos.

2. NODO DE CONTROL ANSIBLE CON SSH A LOS WORKERS

A partir de ahora, prácticamente todo se hace con el master. Tenemos que instalar Ansible en el master, con SSH pass para que las máquinas se puedan comunicar entre ellas de forma segura. Lo instalamos con los siguientes comandos:

```
***: Command not found
blillo@k8s-master:~$ sudo apt update -y && sudo apt upgrade -y && sudo apt install -y ansible sshpass_
```

Una vez tenemos Ansible, el siguiente paso es crear una clave SSH (para que cada vez que haya conexión, no esté preguntando por ella) y enlazar los dos worker con la clave del maestro. Generamos la clave² con:

```
libgami - true
blillo@k8s-master:~$ ssh-keygen -t ed25519 -C "ansible-control"
```

Copiamos la clave criptográfica a todas las máquinas con:

```
+-----[SHA256]-----+
blillo@k8s-master:~$ ssh-copy-id blillo@192.168.56.10 && ssh-copy-id 192.168.56.11 && ssh-copy-id 192.168.56.12
```

Comprobamos que podemos hacer SSH entre las máquinas sin contraseña, y ya podríamos pasar a la parte interesante, que es crear el inventario para que Ansible conecte las máquinas. Podría escribir a mano cada documento en Ubuntu Server, pero al no tener interfaz gráfica, es un poco tedioso.

Mi método es, crear los documentos en mi PC (Windows) con VScode, por ejemplo, crear toda la estructura de carpetas, subirlo a un repositorio en GitHub y desde la máquina virtual master hacer un “git clone” de todo el contenido, matando dos pájaros de un tiro: tengo un seguimiento del trabajo tanto en mi PC como en las máquinas virtuales, y me ahorro mucho tiempo a la hora de escribir código.

9. EVOLUCIÓN Y TRABAJO FUTURO

10. BIBLIOGRAFÍA

² Ed25519 es un algoritmo de encriptación basado en curvas elípticas